

Weightage = {this: low, magazine: high, is: low, great: high, bad: high, quality: high, of: low}

Term	Doc1	Doc2	Doc3
this	1	1	1
magazine	1	1	1
is	1	1	1
great	1	0	1
bad	0	1	0
quality	0	0	1
of	0	0	1

Words like 'magazine', 'great', 'bad', and 'quality' have higher TF-IDF weights because they are important in individual documents.

Step 2: Compute TF-IDF matrix

The components of TF-IDF are:

1. **TF (Term Frequency)**: It measures the frequency of a term within a document.

$$TF(t, d) = \frac{\text{frequency}(t) \text{ within } d}{\text{Total number of terms within } d}$$

2. **IDF (Inverse Document Frequency)**: It measures the uniqueness of the words across the documents.

$$IDF\left(t, \sum_1^N (d)\right) = \log\left(\frac{N}{(1+df(t))}\right)$$

N = total number of documents

df(t) = number of documents containing the term 't'

3. **TF-IDF score**

Combining TF and IDF, the final TF-IDF formula is:

$$TF \cdot IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

4. TF (Term Frequency) table

Term	Doc1 (TF)	Doc2 (TF)	Doc3 (TF)
magazine	1/4	1/4	1/7
great	1/4	0	1/7
bad	0	1/4	0
quality	0	0	1/7

5. IDF table

Term	Appearance in documents	IDF
magazine	d1, d2, d3	-0.12493874
great	d1, d3	0
bad	d2	0.176091259
quality	d3	0.176091259

6. TF-IDF table

Term	Doc1	Doc2	Doc3
magazine	-0.03	-0.03	-0.02
great	0.00	0.00	0.03
bad	0.00	0.04	0.00
quality	0.00	0.00	0.03

'great' and 'quality' have higher TF-IDF values because they are rare and carry information for the classification of these documents. 'magazine' has negative IDF values and is therefore less useful for the classification of these sentiments.

Implementation

Here is a Python implementation of TF-IDF with four documents. The TF-IDF text processing technique can be easily understood by using this code, which generates TF, IDF, and TF x IDF tables. Scikit-learn is the most appropriate source for the TF-IDF library.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from tabulate import tabulate
import pandas as pd
pd.set_option('display.max_columns', None)
# Sample documents
corpus = [
    "this magazine is great",          #Doc1
    "this magazine is bad",           #Doc2
    "the quality of this magazine is great", #Doc3
    "the name of this excellent magazine is OSFY", #Doc4
]
# Create TF-IDF vectorizer object
vectorizer = TfidfVectorizer(stop_words='english', use_idf=True)
# Fit and transform the corpus
vector = vectorizer.fit_transform(corpus)
# Vocabulary
# TF
feature_names = vectorizer.get_feature_names_out()
df = pd.DataFrame(vector.toarray(), columns=feature_names)
# IDF TABLE
# Get IDF values for each term
idf_values = vectorizer.idf_
idf_df = pd.DataFrame({'term': feature_names, 'idf': idf_values})

# Multiply TF x IDF

tf_df = df
```